



Welcome to the Godot Adventure Game Course. In this comprehensive three-part series, you will learn how to create a *Breath of the Wild* inspired game using the Godot engine. This course is designed to help beginners at Zenva understand the fundamentals of 3D game development in Godot. By the end of this series, you will have the skills to create a variety of 3D games.

Course Overview

The course is divided into three parts, each focusing on different aspects of game development:

Part 1: Game Basics

- Setting up a character
- Working with animations using animation trees
- Creating randomized monsters

Part 2: Inventory System

- Creating an inventory to manage weapons and shields
- Connecting the inventory to the main game

Part 3: Level Design

- Creating levels with grass and water shaders for enhanced visuals

Key Learning Outcomes

By the end of the course, you will be able to:

- Create a player character with various actions such as running, jumping, defending, and attacking with different animations
- Implement weapons and shields that the player can hold and change
- Generate skeletons with random appearances and weapons
- Develop a basic fighting system for player and enemy interactions

Overarching Learning Goals

The broader objectives of this course include:

- Understanding 3D game development
- Manipulating attributes of 3D objects
- Working with skeleton animations
- Mastering the animation tree node
- Creating basic AI for enemies and a fighting system

Prerequisites

To get the most out of this course, you should have some basic knowledge of:

- Godot
- GDScript
- 3D game development concepts such as meshes, materials, and importing 3D objects

If you have created any game before in Godot and have experimented with its features, you are well-prepared for this course.



Course Structure

The course is structured to build your skills progressively. Here is what you can expect in each part:

Part 1: Detailed Breakdown

1. **Setting Up a Character:** Learn how to create and configure a player character with basic movements.
2. **Animation Trees:** Understand how to use animation trees to manage complex animations efficiently.
3. **Randomized Monsters:** Discover techniques to generate monsters with varied appearances and behaviors.

Part 2: Detailed Breakdown

1. **Inventory System:** Develop an inventory system to manage weapons and shields, and integrate it with the main game.

Part 3: Detailed Breakdown

1. **Level Design:** Create visually appealing levels using grass and water shaders to enhance the game's aesthetics.

By following this course, you will gain a solid foundation in 3D game development using Godot. Whether you are a beginner or have some experience, this series will equip you with the skills needed to create engaging and visually impressive 3D games.

Course Updated to Godot 4.5

We've updated the project files to Godot version 4.5 for this course – the latest stable release.

Please make sure NOT to use newer versions of the software than what we recommend, as the course material provided might not work as expected.

How to Install Version 4.5

You can download the most recent version of Godot by heading to the Godot website (<https://godotengine.org/>) and clicking the “Download Latest” button on the front page. This will automatically take you to the download page for your operating system.



Once on the download page, just click the option for **Godot 4.5**. This will download the Godot engine to your local computer. From there, unzip the file and simply click on the application launcher – no further installation steps are required!

Download Godot 4 for Windows

**Godot Engine****4.5**

x86_64 - 15 September 2025

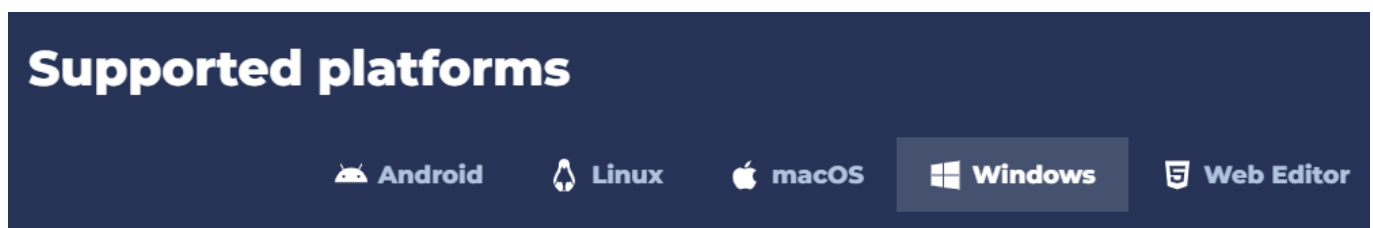
**Godot Engine – .NET****4.5**

x86_64 · C# support · 15 September 2025

Looking for [Godot 3](#), our [experimental releases](#), or a [previous version](#)?



If Godot fails to automatically select the correct operating system for you, you can scroll down to the “Supported platforms” section on the download page to manually select the download version you would like to install:





Welcome to this beginner-friendly guide on getting started with your first Godot project. This article will walk you through the initial setup and understanding of the project structure. By the end, you will be ready to dive into creating your game.

Project Overview

Before we begin, let's familiarize ourselves with the project structure. Your Godot project comes pre-loaded with essential assets and configurations. Here's what you need to know:

- **Graphics:** All the visual elements you will use in your game are already included.
- **Folder Setup:** The project has a basic folder structure to keep things organized.
- **Item Data:** Some data for the items in the game is provided, including scenes and global files.
- **Key Bindings:** Controller and keyboard/mouse key bindings are pre-configured.

You can download the starting file and the completed project from the files tab for this course. Make sure to download the starting file so you can follow along.

Exploring the Project Structure

Let's take a closer look at the project structure in Godot:

1. **Audio Folder:** Contains all the audio files for the game.
2. **Graphics Folder:** Includes subfolders with various graphics. For example, the weapons subfolder contains a dagger graphic.
3. **Scenes Folder:** This folder contains several scenes. Notably, the weapons subfolder has a parent scene called `weapon.tscn` with a `Node3D` and a `RayCast3D`. Other weapon scenes like `sword.tscn` inherit from this parent scene and add specific meshes.
4. **Global Folder:** Contains a scene called `global.tscn` with a script that includes dictionaries storing weapon data. This data includes damage, scene location, animations, range, audio, and more for each weapon.

Input Map Configuration

The project settings include an input map that covers various actions. Here are some key points:

- Most actions have bindings for both the joypad and the keyboard.
- Exceptions include camera panning, which is done via the right joystick on a controller or the mouse on a keyboard setup.
- You can easily customize these bindings to suit your preferences.

Global File

In the globals folder, you will find the global file mentioned earlier. This file, `global.gd`, contains the script with the weapon data dictionaries. This setup will save you a lot of typing later on, even though you won't need it immediately.

Getting Started

Now that you have a clear understanding of the project structure and initial setup, you are ready to begin creating your game. Stay tuned for the next steps in our journey to building an exciting game with Godot.

Happy coding!

Welcome to this beginner-friendly tutorial on getting started with your first project in **Godot**. In this article, we will walk through the initial steps of creating an overworld and a basic player character. By the end of this tutorial, you will have a static scene with a player character and a basic environment. Let's get started!

Setting Up the Project

To begin, we need to create a new 3D scene in Godot. Follow these steps:

1. Create a new 3D scene and rename it to Game.
2. Save the scene inside the Scenes directory, specifically under Scenes/Levels/Game.tscn.

Creating the Terrain

Next, we will add some basic platforms to serve as our terrain:

1. Add a CSGCombiner node and rename it to terrain.
2. Add a CSGBox as a child of the terrain node to create a basic floor.
3. Adjust the size of the CSGBox to create a platform for the player to stand on.

Applying Materials

To make the terrain look nicer, we will apply a material to it:

1. In the Inspector, create a new StandardMaterial.
2. Set the albedo color to a dark gray (4e4e4e).
3. Add a texture from Graphics/Test Texture/White.svg.
4. Save the material as TestMaterial inside the Graphics/Test Texture directory for reuse.

Adding More Terrain Features

You can add more features to your terrain by following these steps:

1. Add additional CSGBox nodes to create more platforms.
2. Apply the saved material to these new platforms.

Setting Up Collision Layers

To ensure proper collision detection, configure the collision layers:

1. Go to Project -> Project Settings -> 3D Physics.
2. Ensure you have collision layers set up for terrain, player, and enemies.
3. Set the terrain layer for the CSGCombiner node and ensure the collision mask is empty.

Creating the Player Character

Now, let's create the player character:

1. Create a new scene with a CharacterBody3D root node and rename it to character.
2. Add a CollisionShape3D with a capsule shape to the character.
3. Save the scene as character.tscn inside the Scenes/Characters directory.

Customizing the Player Character

To customize the player character, follow these steps:



1. Create an inherited scene from `character.tscn` and rename it to `player`.
2. Save the scene as `player.tscn`.
3. Choose a character model (e.g., `rogue.glb`) and add it to the scene.

Cleaning Up the Character Model

To clean up the character model:

1. Create a new scene with a `3DScene` root node and name it `PlayerSkin`.
2. Add the character model (e.g., `rogue.glb`) as a child node.
3. Make the model local to access all nodes and skeleton parts.
4. Remove any unwanted bone attachments, keeping only the necessary meshes.
5. Save the scene as `PlayerSkin.tscn` inside the `Characters` directory.

Adding the Player to the Game Scene

Finally, add the player to the game scene:

1. In the Game scene, add a `Node3D` and rename it to `characters`.
2. Add the player scene as a child to the `characters` node.
3. Set the collision layers for the player to interact with the terrain and enemies.
4. Add a `Camera3D` to view the player character.

Enhancing the Environment

To improve the environment:

1. Add a `Sun` and an `Environment` node to the Game scene.
2. Run the scene to see the improved environment.

Congratulations! You have now set up a basic overworld and player character in Godot. This foundational knowledge will help you as you continue to build and expand your project. Happy coding!



Welcome to this beginner-friendly guide on enhancing your 3D game level in Godot. In this tutorial, we will focus on improving the world environment and lighting to create a more visually appealing game. We will also expand the level to make it more engaging for players. Let's get started!

Setting Up the World Environment

The world environment is crucial for creating a believable and immersive 3D game. Here's how you can set it up:

- **Custom Sky Color:** Navigate to the World Environment node and go to the Environment section. Set a custom color for the sky background. For example, use the hex code #7B82E6 for a purple hue.
- **Ambient Light:** Change the ambient light from background to color. Use the hex code #F7D5DB and set the energy to 0.6 for a more neutral look.
- **Shadows and Glow:** Enable SSAO and SSIL for better shadows. Also, enable glow with an intensity of 0.09 and strength of 0.85. Set the blend mode to screen.
- **Fog Effects:** Enable fog and change the light color to #2DFE3. Lower the density to create a bluish fog that enhances the sky color. Do the same for volumetric fog with a density of 0.005.
- **Adjustments:** Increase brightness, contrast to 1.2, and saturation to 1.05. Be cautious with saturation as it can be easily overdone.

Configuring Camera Attributes

Camera attributes play a significant role in how your game world is perceived. Follow these steps:

1. Enable Depth of Field (DOF) and set it to far enabled.
2. Adjust the far distance to 150 and set the transition and amount to make the effect more subtle.

Adjusting the Directional Light

The directional light is essential for creating realistic lighting in your game. Here's how to adjust it:

- **Light Color:** Change the light color to a warmer tone. For example, use the hex code #F0F0F0.
- **Shadows:** Ensure shadows are enabled. Lower the opacity of the shadows slightly and set the blur to 1.2.

Expanding the Level

A larger and more varied level makes the game more engaging. Here's how to expand your level:

1. Select the platform and hold Alt (Windows) or Command (Mac) to drag and expand it.
2. Move and expand other elements like boxes to create a bigger platform.
3. Duplicate platforms and adjust their positions to create stairs and varied heights.
4. Add a CSGTorus for a donut-shaped object. Rotate and align it to create a jump-through feature. Adjust the inner and outer radius for the desired size.
5. Apply a material to the torus. Create a new standard material, set the albedo, and choose a texture like white.svg. Adjust the color for variance.

Final Touches

Once you have expanded your level and added new elements, move the player to a suitable starting position. This ensures that the player can explore the newly expanded level effectively.



Remember, lighting and level design can be subjective. Experiment with different settings to find what looks best for your game. Happy designing!



In this lesson, we will focus on implementing the player camera in Godot. The camera is a crucial component for player movement, as it determines the direction in which the player moves. We will start by setting up the camera as a physical object that can interact with the environment, and then we will add functionality to control the camera using either a controller or the mouse.

Setting Up the Camera

First, let's set up the camera in the player scene:

1. Remove the existing Camera3D node from the player scene.
2. Create a new scene with a SpringArm3D as the root node. This will act as an extendable and compressible arm for our camera.
3. Set the spring_length property of the SpringArm3D to 6.5 meters.
4. Attach a Camera3D node to the end of the SpringArm3D and set it as the current camera.
5. Rename the SpringArm3D node to CameraController.
6. Save the scene in the Scenes/Characters folder.

Next, add the CameraController to the player scene:

1. Instantiate the CameraController scene as a child of the player scene.
2. Adjust the position of the CameraController to be at a height of 1.4 meters and rotate it down by -25 degrees on the x-axis.

Making the Camera Interactive

To make the camera interactive, we will add a script to the CameraController. Let's start by setting up some default values to control the min and max x the camera can go, as well as how fast the camera moves:

```
extends SpringArm3D
```

```
@export var min_limit_x: float = -0.8
@export var max_limit_x: float = 0.2
@export var horizontal_acceleration: float = 2
@export var vertical_acceleration: float = 1
@export var mouse_acceleration: float = 0.005
```

Adding Controller Input

Now, let's add functionality to control the camera using a controller. We will use the `_process` function to capture input from the right joystick. Specifically, it gets the input vector from the player's controller (tied to our input mapping). It then accelerates it based on the delta time and acceleration values, and then rotates the camera using the `rotate_from_vector` function. This allows for smooth panning of the camera.

```
func _process(delta: float) -> void:
    var joy_dir = Input.get_vector("pan_left", "pan_right", "pan_up", "pan_down")
    var joy_dir_accelerated = joy_dir * delta * Vector2(horizontal_acceleration, vertical_acceleration)
    rotate_from_vector(joy_dir_accelerated)
```



Next, we need to create the `rotate_from_vector` function to handle the rotation of the camera using the vector from the function above. Note that for the x value, we clamp it to make sure the camera doesn't rotate oddly:

```
func rotate_from_vector(v: Vector2):  
    if v.length() == 0: return  
    rotation.y -= v.x  
    rotation.x -= v.y  
    rotation.x = clamp(rotation.x, min_limit_x, max_limit_x)
```

Adding Mouse Input

If you don't have a controller, you can also use the mouse to control the camera. We will use the `_input` function to capture mouse movement and accelerate and rotate it in the same manner we did above:

```
func _input(event: InputEvent) -> void:  
    if event is InputEventMouseMotion:  
        var mouse_dir_accelerated = event.relative * mouse_acceleration  
        rotate_from_vector(mouse_dir_accelerated)
```

To ensure the mouse is captured within the game window, we can set the mouse mode to captured in the `_ready` function:

```
func _ready() -> void:  
    Input.mouse_mode = Input.MOUSE_MODE_CAPTURED
```

Finally, to allow the player to quit the game by pressing the escape key, we can add the following code to the `_input` function:

```
if Input.is_action_just_pressed("ui_cancel"):  
    get_tree().quit()
```

With these steps, you have successfully set up and implemented the player camera in Godot. You can now control the camera using either a controller or the mouse, and the camera will interact physically with the environment.

Going forward, we will focus solely on the controller schema, so feel free to comment out the mouse code if you want. In the next lesson, we will focus on the character movement.



In this lesson, we will focus on implementing player movement in a 3D game using Godot. We will start by setting up the necessary variables and functions in our scripts to handle player input and movement. By the end of this lesson, you will have a player character that can move forward, backward, left, and right based on keyboard or controller input.

Setting Up the Character Script

First, let's create a new script for our character. This script will contain the basic variables and logic needed for movement.

1. Create a new script and attach it to the Character node.
2. Name the script character.gd.
3. Add the following variables to the script:

```
extends CharacterBody3D
```

```
@export var base_speed: float = 4  
var movement_input: Vector2
```

- `base_speed`: This variable determines the base speed at which the character moves.
- `movement_input`: This variable stores the input direction from the player.

Creating the Player Script

Next, we will create a script specifically for the player character. This script will inherit from the `character.gd` script and handle the player's movement logic.

1. Create a new script and attach it to the Player node.
2. Name the script player.gd.
3. Make sure the script inherits from the `character.gd` script by setting the class name to `Character`.
4. Add the following function to the script:

```
extends Character
```

```
func _physics_process(delta: float) -> void:  
    move_logic(delta)  
    move_and_slide()
```

Implementing Movement Logic

Now, let's implement the movement logic in the `player.gd` script. This logic will handle player input and update the character's velocity based on the input direction.

1. Add the following variables to the script:

```
@export var acceleration: float = 8  
@export var deceleration: float = 4
```

- `acceleration`: This variable determines how quickly the character accelerates when moving.



- deceleration: This variable determines how quickly the character decelerates when stopping.

2. Add the following function to the script:

```
func move_logic(delta):
    movement_input = Input.get_vector("left", "right", "forward", "backward").rotated
    (-camera.global_rotation.y)
    var vel_2d = Vector2(velocity.x, velocity.z)

    if movement_input != Vector2.ZERO:
        var speed = base_speed
        vel_2d += movement_input * speed * delta * acceleration
        vel_2d = vel_2d.limit_length(speed)
        velocity.x = vel_2d.x
        velocity.z = vel_2d.y
    else:
        vel_2d = vel_2d.move_toward(Vector2.ZERO, base_speed * delta * deceleration)
        velocity.x = vel_2d.x
        velocity.z = vel_2d.y
```

This `move_logic` function starts by getting the input vector from the player's controller, using the "left", "right", "forward", and "backward" actions. This vector is then rotated to match the camera's orientation, so that the player moves in the direction they are facing.

The code then updates the player's velocity based on the input vector, speed, acceleration, and delta time. If the player is moving, it adds the input vector multiplied by the speed, acceleration, and delta time to the current velocity. It also limits the length of the velocity vector to the maximum speed, to prevent the player from moving too fast while still make the controls feel nice as the player accelerates.

If the player is not moving, the code decelerates the velocity towards zero, using the `move_toward` function. This slows down the player's movement over time, giving a more natural feel to the movement as well.

Finally, the updated velocity is applied to the player's movement, by setting the x and z components of the velocity vector.

Testing the Movement

Finally, let's test the movement to ensure it works as expected.

1. Run the game and try moving the player character using the keyboard or controller.
2. If the player character does not move, ensure that the script is attached to the Player node and that the input map is set up correctly in the project settings.

Congratulations! You have successfully implemented player movement in your 3D game using Godot. In the next lesson, we will build upon this foundation and add more features to our player character.



In this lesson, we will address two key issues with our player's movement in Godot. First, we will ensure that the player's movement is relative to the camera's direction. Second, we will make the player's model rotate in the direction of movement. By the end of this lesson, you will have a player that moves and rotates smoothly relative to the camera's orientation.

Issues with Current Movement

Currently, the player can move right, but this movement is not affected by the camera's orientation. This means that the player moves in the same direction regardless of where the camera is pointing. Additionally, the player's model does not rotate to face the direction of movement.

Making Movement Relative to the Camera

To make the player's movement relative to the camera's direction, we need to rotate the input vector by the camera's rotation. Here's how to do it:

1. First, get a reference to the camera:

```
@onready var camera = $CameraController/Camera3D
```

2. Next, rotate the movement input vector by the camera's rotation:

```
movement_input = Input.get_vector("left", "right", "forward", "backward").rotated(-camera.global_rotation.y)
```

With this change, the player's movement will be relative to the camera's orientation. This means that if you point the camera towards an object and press the forward key, the player will move towards that object.

Rotating the Player's Model

To make the player's rotation feel more natural, we should rotate the player's model to face the direction of movement. Here's how to do it:

1. First, get a reference to the player's skin:

```
@onready var skin = $PlayerSkin
```

2. Next, calculate the target angle based on the movement input:

```
var target_angle = -movement_input.angle() + PI / 2
```

3. Finally, rotate the player's skin towards the target angle:

```
skin.rotation.y = rotate_toward(skin.rotation.y, target_angle, delta * 6)
```

Note that we need to add an extra 90 degrees ($\text{PI} / 2$) to the target angle to ensure that the player faces the correct direction. Also, we need to add the negative sign to the movement input angle to ensure proper rotation.

Summary



- We made the player's movement relative to the camera's orientation by rotating the movement input vector by the camera's rotation.
- We rotated the player's model to face the direction of movement by calculating the target angle based on the movement input and rotating the player's skin towards that angle.

In the next lesson, we will continue to improve the player's movement and add new features to our game. See you there!



In the previous lessons, we enabled basic movement for the player character. However, the movement is still quite limited. To enhance the player's mobility, we will implement jumping and falling mechanics using some projectile math. This lesson will guide you through adding jump functionality to your player character in Godot.

Setting Up Jump Variables

To begin, we need to define several variables that will control the jump behavior. Open your **character** script and add the following export variables:

```
# Define jump height
@export var jump_height: float = 2.25

# Define time to reach the peak of the jump
@export var jump_time_to_peak: float = 0.4

# Define time to descend from the peak of the jump
@export var jump_time_to_descent: float = 0.3
```

These variables allow you to adjust the height of the jump, the time it takes to reach the peak, and the time it takes to descend. Next, we need to calculate the jump velocity, jump gravity, and fall gravity using these variables.

Calculating Jump Velocity and Gravity

Add the following onready variables to your script:

```
# Calculate jump velocity
@onready var jump_velocity: float = ((2.0 * jump_height) / jump_time_to_peak) * -1.0

# Calculate jump gravity
@onready var jump_gravity: float = ((-2.0 * jump_height) / (jump_time_to_peak * jump_time_to_peak))

# Calculate fall gravity
@onready var fall_gravity: float = ((-2.0 * jump_height) / (jump_time_to_descent * jump_time_to_descent))
```

These calculations use the jump height and time variables to determine the velocity and gravity needed for the jump mechanics.

Implementing Jump Logic

Now, let's implement the logic for the jump. Create a new function called `jump_logic` in your **player script**:

```
func jump_logic(delta):
    # Check if the player is on the floor
    if is_on_floor():
```



```
# Check for jump input
if Input.is_action_just_pressed("jump"):
    # Set the velocity to jump velocity
    velocity.y = -jump_velocity

# Apply gravity based on the velocity
var gravity = jump_gravity if velocity.y > 0.0 else fall_gravity
apply_gravity(gravity, delta)
```

This function checks if the player is on the floor and if the jump input is pressed. If both conditions are met, it sets the vertical velocity to the jump velocity. It also applies gravity based on whether the player is ascending or descending.

Applying Gravity

To apply gravity to the player, create the `apply_gravity` function back in the character script, as we'll want this for enemies too:

```
func apply_gravity(gravity, delta):
    velocity.y -= gravity * delta
```

This function adjusts the vertical velocity based on the gravity and delta time.

Testing the Jump

With the jump logic implemented, you can now test the jump functionality. Run your game and use the designated jump buttons (e.g., space bar on the keyboard) to make the player character jump. Adjust the jump variables as needed to achieve the desired jump behavior.

By following these steps, you have successfully implemented jumping and falling mechanics for your player character in Godot. Experiment with the variables to fine-tune the jump to your liking!



In this lesson, we will focus on enhancing the player's movement by incorporating animations. Currently, the player can move around, but the lack of animations makes the movement appear unnatural. To address this, we will utilize Godot's AnimationTree node, which simplifies the management of multiple animations. Although the AnimationTree node is more advanced, we will proceed step-by-step to ensure clarity.

Creating the AnimationTree

First, we need to create an AnimationTree node within the character class since it will be reused extensively. Here's how to set it up:

1. Open the character scene.
2. Add an AnimationTree node as a child of the character node.
3. Save the scene.

Setting Up the AnimationTree in the Player

Next, we need to configure the AnimationTree within the player. This involves adding an animation player. Follow these steps:

1. Open the player scene.
2. Ensure the AnimationTree node is present.
3. Select the player skin node and enable editable children.
4. Select the AnimationTree node and drag the animation player node into the AnimationTree node. The animation player comes with all the necessary animations we'll need for our project.

Configuring the AnimationTree

Now, we need to define the tree root and set up the blend tree. Here's how to do it:

1. Select the AnimationTree node.
2. In the inspector, set the tree root to the AnimationNodeBlendTree.
3. In the new graph, add an animation node by right-clicking on the output field and selecting "Add Animation Node".
4. Connect the animation node to the output.

For example, you can add any animation to test the setup. Make sure to remove the animation node when you're done for the next step.

Creating a State Machine

To manage the animations effectively, we will use a state machine. Follow these steps:

1. Right-click the graph and add a "StateMachine" node.
2. Rename the state machine to MoveStateMachine.
3. Open the state machine editor with its button.
4. Right-click in the graph, choose Add Animation, and from the menu add the Idle animation. This will create a new Idle node.
5. Click the Connect Nodes button in the top menu and connect the start node to the new Idle node.
6. Back in the Tree Root, make sure to connect the State Machine to the Output node.

To ensure the idle animation loops continuously, follow these steps:



1. Select the player skin node.
2. Find the idle animation in the animation player.
3. Enable animation looping.

Feel free to test it out in game to ensure it's working.

Adding Running Animation

Next, we will add a running animation and manage the transitions between idle and running states. Follow these steps:

1. In the MoveStateMachine, add a running animation node (e.g., "Running_A").
2. Disable the auto-transition button.
3. Create transitions between the idle and running nodes so they can transition between each other.

To manage these transitions in the script, follow these steps:

1. Open the character script.
2. At the top, get the MoveStateMachine node:

```
@onready var move_state_machine = $AnimationTree.get("parameters/MoveStateMachine/playback") as AnimationNodeStateMachinePlayback
```

3. Create a function to switch between states:

```
func set_move_state(state_name: String):  
    move_state_machine.travel(state_name)
```

4. In the player script, use the set_move_state function to switch between idle and running states based on movement input.

```
if movement_input != Vector2.ZERO:  
    ...  
    set_move_state('Running_A')  
else:  
    ...  
    set_move_state('Idle')
```

Looping the Running Animation

To ensure the running animation loops continuously, follow these steps:

1. Select the player skin node.
2. Find the running animation in the animation player.
3. Enable animation looping.

Adding Transition Times

To make the transitions between animations smoother, we can add transition times. Follow these steps:

1. In the MoveStateMachine, select the transitions between idle and running nodes.



2. Set the XFadeTime to 0.2 for all transitions.

Adding Jump Animation

Finally, we will add a jump animation. Follow these steps:

1. In the MoveStateMachine, add a jump animation node (e.g., "Jump_Idle").
2. Create transitions between idle, running, and jump nodes (there will be 4).
3. Set the XFadeTime to 0.2 for all new transitions.
4. In the player script, use the set_move_state function to switch to the jump state when the player is in the air:

```
func jump_logic(delta):
    if is_on_floor():
        if Input.is_action_just_pressed("jump"):
            velocity.y = -jump_velocity
    else:
        set_move_state('Jump_Idle')
    var gravity = jump_gravity if velocity.y > 0.0 else fall_gravity
    apply_gravity(gravity, delta)
```

By following these steps, you will have successfully integrated animations into the player's movement, making it look more natural and engaging.



In this lesson, we will delve into the exciting world of weapons and attacks in Godot. By the end of this lesson, you will have a solid understanding of how to manage multiple weapons, toggle between them, and equip them to your player character. We will also cover how to set up animations for these weapons and ensure that the player can only damage opponents when attacking.

Understanding Weapon Data

Before we dive into the code, let's understand the data structure we will be working with. In Godot, we use dictionaries to store information about our weapons. Each weapon has a unique set of properties such as type, damage, scene, animation, range, and sound.

Here is an example of how the weapon data is structured in global.gd:

```
var weapons: Dictionary = {
    'dagger':
    {
        'type': 'weapon',
        'damage': 1,
        'scene': preload("res://scenes/weapons/dagger.tscn"),
        'animation': '1H_Melee_Attack_Stab',
        'range': 1.2,
        'audio': preload("res://audio/dagger_sound.wav")
    },
    'sword':
    {
        'type': 'weapon',
        'damage': 2,
        'scene': preload("res://scenes/weapons/sword.tscn"),
        'animation': '1H_Melee_Attack_Slice_Horizontal',
        'range': 1.5,
        'audio': preload("res://audio/sword_sound.wav")
    },
    'axe':
    {
        'type': 'weapon',
        'damage': 3,
        'scene': preload("res://scenes/weapons/axe.tscn"),
        'animation': '2H_Melee_Attack_Spin',
        'range': 1.3,
        'audio': preload("res://audio/axe_sound.wav")
    },
    'staff':
    {
        'type': 'weapon',
        'damage': 1,
        'scene': preload("res://scenes/weapons/staff.tscn"),
        'animation': '2H_Melee_Attack_Slice',
        'range': 2.1,
        'audio': preload("res://audio/staff_sound.wav")
    },
}
```

Equipping Weapons



To equip a weapon to the player character, we need to attach it to the character's skeleton. Godot provides a special node called BoneAttachment3D for this purpose. Here's how you can set it up:

1. Add a BoneAttachment3D node to the Skeleton3D node of your player character.
2. In the inspector, attach the BoneAttachment3D node to the desired bone, such as handslot.r for the right hand.
3. Rename the BoneAttachment3D node to something meaningful, like RightHand.

Dynamic Weapon Equipping

To make the weapon equipping dynamic, we need to write some script. We will create a function to equip weapons based on the data from our dictionary.

First, in the player script, let's declare the variables for our weapons and the current weapon index:

```
var weapons = [Global.weapons['sword']]
var weapon_index: int = 0
```

Next, we create the `_ready` function to equip the default weapon when the scene is ready:

```
func _ready() -> void:
    equip(weapons[weapon_index], $PlayerSkin/Rogue/Rig/Skeleton3D/RightHand)
```

Next, we'll make a new function for equipping weapons in the **character script**. The equip function is responsible for attaching the weapon to the specified slot. It does so by first clearing all the children, instantiating the the model scene, and adding that as a child to the slot:

```
func equip(data, slot):
    for child in slot.get_children():
        child.queue_free()
    var item_scene = data['scene'].instantiate()
    slot.add_child(item_scene)
```

You can run the game and test this out. Now the player should be holding the sword model. You can changes the weapon in the weapons variable to try out different weapons from the dictionary!

Conclusion

In this lesson, we covered how to manage and equip weapons dynamically in Godot. We explored the data structure for weapons, set up the BoneAttachment3D node, and wrote the necessary script to equip weapons. In the next lesson, we will dive deeper into weapon animations and attacks.



In this lesson, we will focus on implementing the attack functionality for the player character using different weapons. Each weapon will have its unique attack animation, and we will ensure that the player can only attack when not already in the middle of an attack animation. By the end of this lesson, you will have a player character that can attack using various weapons, each with its distinct animation.

Setting Up the Weapon Script

First, we need to create a script for the weapon. This script will handle the weapon's properties such as animation, damage, and radius. Follow these steps:

1. Create a new script and attach it to the weapon parent scene.
2. Define the necessary variables in the script. We need to know whether its a weapon or shield, the animation string, the damage, its parent, and its attack radius:

```
extends Node3D
```

```
const equipment_type = 'Weapon'  
var animation: String  
var damage: int  
var parent  
var radius: float
```

3. Create a constructor function to set up these variables:

```
func setup(weapon_animation, weapon_damage, weapon_radius, weapon_parent):  
    animation = weapon_animation  
    damage = weapon_damage  
    parent = weapon_parent  
    radius = weapon_radius
```

Equipping the Weapon

Next, we need to modify the equip function in the character.gd script to set up the weapon when it is equipped. This involves calling the setup function we just created and setting the attack animation.

1. Open the character.gd script.
2. Modify the equip function to include the following code. In this case, all the data will come from our dictionary, so we merely need to pass in the correct key names from the data:

```
func equip(data, slot):  
    for child in slot.get_children():  
        child.queue_free()  
    var item_scene = data['scene'].instantiate()  
    slot.add_child(item_scene)  
    if data['type'] == 'weapon':  
        item_scene.setup(data['animation'], data['damage'], data['range'], self)
```

Adding the Attack Animation



To play the attack animation, we need to add a one-shot node to the animation tree. This node will play the attack animation once and then revert to the default animation.

1. Open the animation tree for the player character.
2. In the Tree Root, create a new Animation node and name it "AttackAnimation."
3. Add a new one-shot node in the Tree Root and name it "AttackOneShot"
4. In the Inspector, set the fade in and fade out times for the OneShot node to 0.2 (or another value that looks good).
5. Connect the StateMachine to the "in" input node of the OneShot node, and connect the Animation node to the "shot" entry point.
6. Connect the OneShot node to the Output.

Setting up the Attack Animation

In order to use our new one-shot for attacking, we need to do some configuration. In the **character script**, we need to get a reference to our attack animation node.

```
@onready var attack_animation = $AnimationTree.get_tree_root().get_node('AttackAnimation') as AnimationNodeAnimation
```

Now, we can alter the equip function to target this node (and inevitably use it to play our animations).

```
func equip(data, slot):
    for child in slot.get_children():
        child.queue_free()
    var item_scene = data['scene'].instantiate()
    slot.add_child(item_scene)
    if data['type'] == 'weapon':
        item_scene.setup(data['animation'], data['damage'], data['range'], self)
        attack_animation.animation = data['animation']
```

Playing the Attack Animation

Now, we need to create a function to play the attack animation when the player presses the attack button. This function will also ensure that the player cannot attack multiple times in quick succession.

1. Open the player.gd script.
2. Create a new function called `ability_logic`. In this function, we will check if we've pressed the attack input, and if so try to play our animation:

```
func ability_logic():
    if Input.is_action_just_pressed("attack"):
        $AnimationTree.set("parameters/AttackOneShot/request", AnimationNodeOneShot.ONE_SHOT_REQUEST_FIRE)
```

3. Call this function in the `_physics_process` function before `move_and_slide`:

```
func _physics_process(delta: float) -> void:
```



```
move_logic(delta)
jump_logic(delta)
ability_logic()
move_and_slide()
```

Preventing Multiple Attacks

To prevent the player from attacking multiple times in quick succession, we need to set an attacking variable to true when the attack animation starts and to false when it ends. This will make sure we don't overlap our attacks constantly.

1. In the character.gd script, add a new variable:

```
var attacking: bool = false
```

2. Back inside the player, modify the ability_logic function to set attacking to true when the attack animation starts (and also make sure we don't process anything if the variable is true):

```
func ability_logic():
    if Input.is_action_just_pressed("attack") and not attacking:
        $AnimationTree.set("parameters/AttackOneShot/request", AnimationNodeOneShot.0
NE_SHOT_REQUEST_FIRE)
        attacking = true
```

3. Connect the animation_finished signal of the animation tree to a new function in the character.gd script. Then, use this signal to set the variable back to false when the animation is finished:

```
func _on_animation_tree_animation_finished(anim_name: StringName) -> void:
    if 'Attack' in anim_name:
        attacking = false
```

With these steps, you should have a player character that can attack using different weapons, each with its unique animation. The player will only be able to attack once per animation cycle, preventing multiple attacks in quick succession.



In this lesson, we will introduce shields to our game and implement the functionality for the player to equip and use them. This process is similar to how we handled weapons, allowing us to reuse some of the existing logic. By the end of this lesson, the player will be able to defend against attacks using a shield.

Shield Scenes

Similar to weapons, shields are also organized into scenes. We have a parent scene named `shield.tscn` which serves as the base for multiple shield types:

- `round_shield.tscn`
- `square_shield.tscn`
- `spike_shield.tscn`

Each shield scene contains a mesh and inherits from the parent shield scene. This structure allows us to manage different shield types efficiently.

Global Dictionary for Shields

In the global dictionary, we have a shields dictionary that stores information about each shield, including its type, defense value, and associated scene. Here is an example of how the shields are defined in the global dictionary:

```
var shields: Dictionary = {
    'square': {
        'type': 'shield',
        'defense': 0.8,
        'scene': preload("res://scenes/shields/square_shield.tscn")
    },
    'round': {
        'type': 'shield',
        'defense': 0.9,
        'scene': preload("res://scenes/shields/round_shield.tscn")
    },
    'spike': {
        'type': 'shield',
        'defense': 0.6,
        'scene': preload("res://scenes/shields/spike_shield.tscn")
    },
}
```

Equipping Shields

To equip shields, we need to modify the player script. Follow these steps:

1. Add a variable to store the shields array and the current shield index:

```
var shields = [Global.shields['round']]
var shield_index: int
```

2. Create a bone attachment for the shield in the player's skeleton. Name this node `LeftHand` and attach it to the `handslot_left` bone (similar to what was done for the weapon).



3. Equip the shield using the equip function, similar to how we equipped weapons:

```
func _ready() -> void:
    ...
    equip(shields[shield_index], $PlayerSkin/Rogue/Rig/Skeleton3D/LeftHand)
```

Shield Script

Next, we want to be able to add some data to our shield. Create a script for the shield to handle its defense value and other properties. Here is an example of the shield script:

```
extends Node3D
```

```
var defense: float
const equipment_type = 'Shield'
```

In the equip function, we now need to add that data to the shield. For the shield, we only really need to add the defense.

```
func equip(data, slot):
    for child in slot.get_children():
        child.queue_free()
    var item_scene = data['scene'].instantiate()
    slot.add_child(item_scene)
    if data['type'] == 'weapon':
        item_scene.setup(data['animation'], data['damage'], data['range'], self)
        attack_animation.animation = data['animation']
    if data['type'] == 'shield':
        item_scene.defense = data['defense']
```

Defending with Shields

To use the shield for defending, we need to add input handling and animation blending. Follow these steps:

1. Add a variable to store the defending state in the **character script**:

```
var defending: bool = false
```

2. Update the defending state based on player input. This will essentially create a toggle based on whether the input is being pressed or not:

```
func ability_logic():
    defending = Input.is_action_pressed("defend")
```

Animation Blending

We next need to work on the animation blending. In the player's animation tree, add a new *Animation* node called "Defend Animation". When defending, we want this to be blended into



the moving animation. This way the player can still walk while showing the shield animation. To do this:

1. Create a new node called *Blend2* and rename it to “DefendBlend”.
2. Once created, connect the *MoveStateMachine* node to the “in” input, and the *DefendAnimation* node into the “blend” input.
3. Finally, connect the *Blend2* output to the “in” field of the *OneShot* node.
4. In the *Blend2* mode, select *Edit Filters* and select all parts that pertain to the upper body. Anything pertaining to the legs and below should be unchecked. This is what will allow us to apply our animation selectively.
5. While here, select *Edit Filters* for the *OneShot* node and repeat the selection. This will allow us to also blend the attack animation – allowing the player to attack while walking as well.

By following these steps, you will have successfully set up shields for your game. In the next lesson, we will continue improving our shields to work more smoothly in our game.

In this lesson, we will focus on implementing the defending mechanism for the player character in our game. Currently, the defending feature is not functional; pressing the defend button does not trigger any action. Our goal is to toggle the defending state and create a smooth transition between defending and not defending using tweens.

Objective

By the end of this lesson, you will be able to:

- Toggle the defending state of the player character.
- Create and use tweens to animate the transition between defending and not defending.
- Update the animation tree to reflect the defending state.

Step-by-Step Implementation

1. Detecting Changes in Defending State

First, we need to detect when the defending state changes. We will use the `setValue` function on the defending variable (in the character script) to check for changes in the defending state and change it accordingly. Note that this will use a function we'll create next.

```
var defending: bool = false:
set(value):
    if not defending and value:
        defend_toggle(true)
    if defending and not value:
        defend_toggle(false)
    defending = value
```

2. Toggling the Defending State

Next, still in the character script, we will create a function to toggle the defending state. This function will be called whenever the defending state changes. We will use tweens to smoothly transition in and out of the state. We'll set up a function that changes our animation tree accordingly as well based on the tween.

```
func defend_toggle(forward: bool):
    var tween = create_tween()
    tween.tween_method(_defend_change, 1.0 - float(forward), float(forward), 0.25)

func _defend_change(value):
    $AnimationTree.set("parameters/DefendBlend/blend_amount", value)
```

3. Testing the Implementation

Finally, we will test our implementation to ensure that the defending mechanism works as expected. Pressing the defend button should toggle the defending state and animate the transition smoothly.

Conclusion



In this lesson, we implemented the defending mechanism for the player character. We detected changes in the defending state, toggled the defending state, and created a smooth transition using tweens. By following these steps, you should now have a functional defending mechanism in your game.

In the next lesson, we will add one more equipment item.



In this lesson, we will enhance the player character by adding visual customization options, implementing weapon and shield switching, and incorporating multiple movement speeds. These additions will make the player character more dynamic and engaging. Let's break down each enhancement step by step.

Adding Visual Customization

First, let's add some visual customization options for the player character. These options will be purely visual and will not affect gameplay. If you open the `global.gd` script, you will find this data prepared like the weapons and shield in a dictionary:

```
var style: Dictionary = {
    'sunglasses': {
        'type': 'style',
        'scene': preload("res://scenes/style/sunglasses.tscn"),
    },
    'starglasses': {
        'type': 'style',
        'scene': preload("res://scenes/style/starglasses.tscn"),
    },
    'duckhat': {
        'type': 'style',
        'scene': preload("res://scenes/style/duck_hat.tscn"),
    },
    'tophat': {
        'type': 'style',
        'scene': preload("res://scenes/style/tophat.tscn"),
    },
}
```

Next, open the player scene and add a `BoneAttachment3D` node as a child of the player's skeleton. Target the head bone and rename the `BoneAttachment3D` node to `Head`.

In the player script (`player.gd`), add an array for styles and an index to keep track of the current style:

```
var styles = [Global.style['duckhat']]
var style_index: int
```

Finally, in the `_ready` function, equip the style as has been done with other options:

```
func _ready() -> void:
    equip(weapons[weapon_index], $PlayerSkin/Rogue/Rig/Skeleton3D/RightHand)
    equip(shields[shield_index], $PlayerSkin/Rogue/Rig/Skeleton3D/LeftHand)
    equip_style(styles[style_index], $PlayerSkin/Rogue/Rig/Skeleton3D/Head)
```

Switching Weapons and Shields

Next, let's implement the ability to switch between different weapons and shields during gameplay.



First, we need to add more weapon options for the player. In the player script, alter the arrays for weapons and shields to include more from our dictionary collections:

```
var weapons = [Global.weapons['sword'], Global.weapons['dagger'], Global.weapons['staff']]
var weapon_index: int
var shields = [Global.shields['round'], Global.shields['square'], Global.shields['spike']]
var shield_index: int
```

Next, we need a way to change the index to cycle through the array. Let's add an `_input` function to handle weapon and shield switching. In this function, we need to check for our switch inputs (from the input mapping). We then change the index and call the `equip` function again to change the equipment. The `posmod` function seen below ensures that the index wraps around to the beginning of the array when it exceeds the array size.

```
func _input(_event: InputEvent) -> void:
    if Input.is_action_just_pressed("switch_weapon"):
        weapon_index = posmod(weapon_index + 1, weapons.size())
        equip(weapons[weapon_index], $PlayerSkin/Rogue/Rig/Skeleton3D/RightHand)
    if Input.is_action_just_pressed("switch_shield"):
        shield_index = posmod(shield_index + 1, shields.size())
        equip(shields[shield_index], $PlayerSkin/Rogue/Rig/Skeleton3D/LeftHand)
```

If you test this out in the game, you should be able to cycle through the weapons and shields in the game now. You'll even see how our modular system easily allows each weapon to work with the animation system!

Implementing Multiple Movement Speeds

Finally, let's implement multiple movement speeds for the player character. The player should move faster when running and slower when using a shield.

In the `character.gd` script, add export variables for run speed and defend speed:

```
@export var run_speed: float = 6
@export var defend_speed: float = 2
```

Then, update the `move_logic` function to use different speeds based on the player's actions:

```
func move_logic(delta):
    ...

    if movement_input != Vector2.ZERO:
        var speed = run_speed if Input.is_action_pressed("run") else base_speed
        speed = defend_speed if defending else speed
        vel_2d += movement_input * speed * delta * acceleration
        vel_2d = vel_2d.limit_length(speed)
        ...
```



With these enhancements, the player character now has visual customization options, the ability to switch weapons and shields, and multiple movement speeds. These additions make the player character more engaging and dynamic, improving the overall gameplay experience.



In this article, we want to issue a challenge. Now that you've seen how the player is created, try to create the enemy on your own! We will go over this step-by-step in the next few lessons, but the best way to learn is to practice on your own.

Steps to Create Enemy Skeletons

1. Create an Enemy Class

Begin by creating a new enemy class that inherits from the character class. Choose one of the skeleton models you prefer for this task.

2. Implement Enemy Behavior

- **Walk Towards the Player:** The enemy should start moving towards the player when the player is within a certain radius. You can achieve this by checking the distance between the enemy and the player in the enemy's update function.
- **Attack the Player:** Once the player is close enough, the enemy should initiate an attack. The range and animation for the attack should be retrieved from the weapon dictionary, similar to how it was done for the player.

3. Animate the Enemy

- Ensure that all actions, such as walking and attacking, are properly animated. Use the animation tree to manage these animations efficiently.
- **Trigger the Attack:** To trigger the attack at intervals, use a timer. This method provides a straightforward way to control the frequency of enemy attacks.

Tips for Success

- Take your time with this exercise. It involves multiple steps and logic that you can reuse from the player character setup.
- Pause and review the video instructions as needed. Break down the task into smaller, manageable parts to ensure you understand each step.

This exercise not only enhances your game but also reinforces your understanding of character behavior and animation in Godot. Happy coding and we'll see you in the next video with the start of our solution!



In this lesson, we will set up an enemy class in Godot and program it to move and rotate towards the player. This involves creating a new scene for the enemy, adding a skin, and scripting the enemy's behavior. By the end of this lesson, you will have an enemy that can follow and face the player within a specified radius.

Setting Up the Enemy Scene

To begin, let's create a new scene for the enemy. Follow these steps:

1. Close any unnecessary scenes in Godot, such as the player, player skin, and duck head.
2. Create a new inherited scene from the Character class and rename it to skeleton.tscn.

Adding a Skin to the Enemy

Next, we need to add a skin to the enemy so that it has a visible representation in the game. Here's how to do it:

1. Create a Node3D and rename it to Skins.
2. Add one of the imported skins (e.g., skeleton_warrior.glb) as a child to the Skins node.
3. Make the scene local so that you can edit it.

Scripting the Enemy Behavior

Now, let's add a script to the enemy that will control its movement and rotation towards the player. Follow these steps:

1. Remove the existing Character script from the enemy.
2. Add a new script named skeleton.gd.
3. Ensure the script extends Character instead of CharacterBody3D.
4. Add our starting variables. In this case we need to store a reference to the player, a radius which determines how far away the skeleton can see the player, and the speed it moves. We want the skeleton to move a bit slower than the player by default.

Here is the initial setup for the skeleton.gd script:

```
extends Character
```

```
@onready var player = get_tree().get_first_node_in_group('Player')
@export var notice_radius: float = 30
@export var speed: float = 3
```

Implementing Movement and Rotation

To make the enemy move and rotate towards the player, we need to implement the following logic:

1. Check if the player is within the notice radius.
2. Calculate the direction towards the player.
3. Move the enemy towards the player.
4. Rotate the enemy to face the player. The rotation will look familiar as its the same calculations we did for the player.

We will implement most of this logic in the skeleton.gd script in its own function. We can then call



the function, as well as *move_and_slide()* in the *_physics_process* function:

```
func _physics_process(delta: float) -> void:
    move_to_player(delta)
    move_and_slide()

func move_to_player(delta):
    if position.distance_to(player.position) < notice_radius:
        var target_dir = (player.position - position).normalized()
        var target_vec2 = Vector2(target_dir.x, target_dir.z)
        var target_angle = -target_vec2.angle() + PI / 2
        rotation.y = rotate_toward(rotation.y, target_angle, delta * 6)
        velocity = Vector3(target_vec2.x, 0, target_vec2.y) * speed
```

Testing the Enemy Behavior

To test the enemy behavior, follow these steps:

1. Go back to the game scene.
2. Add the *skeleton.tscn* as a child to the character node.
3. Position the enemy in the scene.
4. Run the game and observe the enemy's movement and rotation towards the player.

If you encounter any errors, ensure that the animation tree and player group are set up correctly. The enemy should now move and rotate towards the player within the specified notice radius.

Congratulations! You have successfully set up an enemy class in Godot and programmed it to move and rotate towards the player. In the next lesson, we will work on the animations for the skeleton.



In this lesson, we will focus on enhancing the enemy character in our game. Specifically, we will animate the enemy, randomize its appearance, and equip it with a random weapon. This will make our enemy characters more dynamic and engaging. Let's break down the steps involved:

Animating the Enemy

First, we need to set up the animations for the enemy. We will use the AnimationTree to manage the enemy's animation states. Here's how we can do it:

1. Open the AnimationTree and navigate to the MoveStateMachine.
2. Add two new states: Idle and Running_A. These states will represent the enemy's idle and running animations, respectively.
3. Ensure that the enemy automatically transitions to the Idle state and can switch between Idle and Running_A as needed.
4. Set the XFadeTime to 0.2 for smooth transitions between animations.
5. On the OneShot node in the main tree, set the Fadein and Fadeout timers to 0.2 as well.

Before continuing, make sure to go to the skeleton's animation set and that the Idle and Running animations are set to loop. This way we will get the appropriate behavior.

Updating the Enemy Script

Next, we need to update the enemy's script to control these animations based on the enemy's state. Open the skeleton.gd script and modify it to handle the animation states. We will use the `set_move_state` function from the parent class to update the animation state based on the enemy's movement.

Here is the relevant code snippet from skeleton.gd:

```
func move_to_player(delta):
    if position.distance_to(player.position) < notice_radius:
        ...
        set_move_state('Running_A')
```

Randomizing the Enemy's Appearance

To make the enemy more varied, we will randomize its skin. We have three skins available: `skeleton_warrior`, `skeleton_rogue`, and `skeleton_minion`.

To set this up, we need to go to the Enemy scene and add all three of our meshes as children to the **Skins** node. Make sure to make all the new meshes editable locally, as we'll need that later to equip items. Also, for the new models, make sure their Idle and Running animations are set up to loop.

Now we can work in our code. In the skeleton script, we'll want to add a `_ready` function. In this function, we'll first hide all the children of the *Skins* node to ensure nothing is visible when the random skin is selected. We can then pick a random child and show it.

We also need to ensure that the animation player is correctly set for the chosen skin. The path to the



animation player should be updated dynamically based on the selected skin.

Here is how we can randomize the enemy's skin in the skeleton.gd script:

```
func _ready() -> void:
    for child in $Skins.get_children():
        child.hide()
    skin = $Skins.get_children().pick_random()
    skin.show()
    $AnimationTree.anim_player = "../Skins/" + skin.name + '/AnimationPlayer'
```

Equipping a Random Weapon

Finally, we will equip the enemy with a random weapon. We will reuse the equip function from the parent class to attach the weapon to the enemy's hand. For this to work, we will need to go to each of our models and add a bone attachment targeting the right hand. Then, in the `_ready` function, we can select a random weapon from our global dictionary, and equip that weapon using our parent function.

Here is the code snippet to equip a random weapon:

```
func _ready() -> void:
    var random_weapon = Global.weapons[Global.weapons.keys().pick_random()]
    equip(random_weapon, skin.get_node('Rig/Skeleton3D/RightHand'))
```

Testing the Enemy

After implementing these changes, test the game to ensure that the enemy animations, skin randomization, and weapon equipping are working correctly. You should also see the enemy move towards the player, but not that we have not yet handled state switching just yet.

In the next lesson, we will finish up our skeleton by giving them better state switching and preparing them to attack the player.



In this lesson, we will implement the attack logic for the skeleton enemy in our game. This involves setting up the attack radius, configuring the attack timer, and ensuring the skeleton falls to the ground due to gravity. By the end of this lesson, the skeleton will be able to attack the player when within range and will fall to the ground if it is in the air.

Setting Up the Attack Radius

First, we need to define the attack radius for the skeleton. This radius determines how close the skeleton needs to be to the player before it starts attacking. We will set a default value for the attack radius and then override it based on the weapon the skeleton is equipped with.

In the skeleton script, add the following variable to define the attack radius:

```
var attack_radius: float = 3
```

Next, we need to override this value based on the weapon the skeleton is equipped with. We will add two more variables to the character.gd script to keep track of the current weapon and shield:

```
var current_weapon: Node3D
var current_shield: Node3D
```

These values will be assigned when the skeleton equips a weapon. We will modify the equip function in the character.gd script to set these values:

```
func equip(data, slot):
    for child in slot.get_children():
        child.queue_free()
    var item_scene = data['scene'].instantiate()
    slot.add_child(item_scene)
    if data['type'] == 'weapon':
        item_scene.setup(data['animation'], data['damage'], data['range'], self)
        attack_animation.animation = data['animation']
        current_weapon = item_scene
    if data['type'] == 'shield':
        item_scene.defense = data['defense']
        current_shield = item_scene
```

Now, back in the **skeleton.gd** script, we can set the attack radius based on the current weapon's range in the **_ready** function.

```
attack_radius = current_weapon.radius
```

Moving Towards the Player

Next, we will modify the move_to_player function to stop the skeleton from moving once it is within the attack radius. We will add a condition to check if the player is within the attack radius and stop the skeleton if it is. We can also make sure the animation switches appropriately:



```
func move_to_player(delta):
    if position.distance_to(player.position) > attack_radius:
        velocity = Vector3(target_vec2.x, 0, target_vec2.y) * speed
        set_move_state('Running_A')
    else:
        velocity = Vector3.ZERO
        set_move_state('Idle')
```

Setting Up the Attack Timer

To make the skeleton attack at regular intervals, we will add a timer. We will create a new node called Timers in the skeleton scene and add a **timer node** as a child of Timers. We will set the default wait time to 2.0 seconds, enable Autostart, and connect the timeout signal to a new function called `_on_attack_timer_timeout`.

In the skeleton script, add the following function to handle the attack timer timeout. This function checks if the skeleton is close enough and initiates the attack animation and variable toggle:

```
func _on_attack_timer_timeout():
    if position.distance_to(player.position) < attack_radius:
        $AnimationTree.set('parameters/AttackOneShot/request', AnimationNodeOneShot.0
NE_SHOT_REQUEST_FIRE)
        attacking = true
```

To make the attack interval more organic, we will set a random value for the timer on every attack. We will use the RandomNumberGenerator to generate a random value between 2.0 and 3.5 seconds and reset the timer at the beginning of our callout:

```
var rng = RandomNumberGenerator.new()

func _on_attack_timer_timeout():
    $Timers/AttackTimer.wait_time = rng.randf_range(2.0, 3.5)
    if position.distance_to(player.position) < attack_radius:
        $AnimationTree.set('parameters/AttackOneShot/request', AnimationNodeOneShot.0
NE_SHOT_REQUEST_FIRE)
        attacking = true
```

Applying Gravity

Finally, we will ensure the skeleton falls to the ground due to gravity if it is in the air. We will modify the `_physics_process` function to apply gravity if the skeleton is not on the floor:

```
func _physics_process(delta: float) -> void:
    move_to_player(delta)
    if not is_on_floor():
        apply_gravity(fall_gravity, delta)
    else:
        velocity.y = 0
    move_and_slide()
```



We will use the `apply_gravity` function from the `character.gd` script to apply gravity to the skeleton:

```
func apply_gravity(gravity, delta):  
    velocity.y -= gravity * delta
```

We will pass the `fall_gravity` value from the `character.gd` script to the `apply_gravity` function as seen above.

Conclusion

In this lesson, we implemented the attack logic for the skeleton enemy. We set up the attack radius, configured the attack timer, and ensured the skeleton falls to the ground due to gravity. The skeleton will now attack the player when within range and will fall to the ground if it is in the air.

In the next lesson, we will start working on combat functionality so our attacking is more than just animations.

In this lesson, we will focus on implementing the damage and hit mechanics for both the player and the skeleton characters in our game. By the end of this lesson, you will have a basic attack and hit functionality working in your game. Let's break down the steps involved:

Setting Up the Character Class

First, we need to create two essential functions in the Character class: `attack_logic` and `hit`. These functions will handle the logic for attacking and taking damage.

Creating the attack_logic Function

The `attack_logic` function will check if the character is currently attacking and if the attack hits another character. Here's how you can set it up:

```
func attack_logic():
    if attacking:
        var collider = current_weapon.get_collider()
        if collider and collider != self and "hit" in collider:
            collider.hit(current_weapon)
```

In this function:

- We first check if the character is attacking.
- We then get the collider from the current weapon using the `get_collider` function.
- We ensure the collider exists, is not the character itself, and has a hit function.
- If all conditions are met, we call the hit function on the collider, passing the current weapon as an argument.

Creating the Hit Timer

Before we script our hit function, we need to make use of Timers again to ensure we cannot hit the character multiple times in a single attack. In the **Character** parent scene, create an empty node called "Timers" and add a Timer node as a child. Name this Timer "HitTimer" and toggle the OneShot option on it.

Creating the hit Function

The hit function will handle the logic for when a character is hit by an attack. Here's how you can set it up:

```
func hit(attacking_weapon):
    if not $Timers/HitTimer.time_left:
        if current_shield and defending:
            print('character is defending: ', attacking_weapon.damage * current_shield.defense)
        else:
            print('character was hit: ', attacking_weapon.damage)
            $Timers/HitTimer.start()
```



In this function:

- We check if the HitTimer is not running.
- If the character has a shield and is defending, we calculate the damage based on the shield's defense value. For now we'll just print the value.
- If the character is not defending, we apply the full damage.
- We then start the HitTimer again.

Integrating with Player and Skeleton Classes

Next, we need to ensure that the `attack_logic` function is called in the physics process of both the player and skeleton classes.

Player Class

In the player class, add the following line to the `_physics_process` function:

```
func _physics_process(delta: float) -> void:
    move_logic(delta)
    jump_logic(delta)
    ability_logic()
    move_and_slide()
    attack_logic()
```

Skeleton Class

In the skeleton class, add the following line to the `_physics_process` function:

```
func _physics_process(delta: float) -> void:
    move_to_player(delta)
    if not is_on_floor():
        apply_gravity(fall_gravity, delta)
    else:
        velocity.y = 0
    move_and_slide()
    attack_logic()
```

Testing the Implementation

With all the above steps completed, you should now have a basic attack and hit functionality in your game. Test the implementation by attacking and defending against the skeleton characters to ensure everything is working as expected.

In the next part, we will work on making the attack and hit functionality more visible and refined.



In this lesson, we will enhance the visual feedback of our game by adding hit animations for characters and a flashing effect for shields when defending. These visual cues will make the game more engaging and provide clear feedback to the player. We will achieve this by implementing a basic hit animation and creating a simple shader for the shield flash effect.

Character Hit Animation

The hit animation will scale the character up briefly and then return to the normal size. This effect, known as “squash and stretch,” will give a visual indication that the character has been hit.

Implementing Squash and Stretch

To implement the squash and stretch effect, follow these steps:

Open the character.gd script.

Add a new variable called `squash_and_stretch` with a default value of 1.0. We will then use the **set(value)** function to dynamically set the values that the character should scale.

```
var squash_and_stretch: float = 1.0
set(value):
    squash_and_stretch = value
    var negative = 1.0 + (1.0 - squash_and_stretch)
    skin.scale = Vector3(negative, squash_and_stretch, negative)
```

Standardize the skin variable in the character.gd script to ensure consistency across different character types.

```
var skin: Node3D
```

In the player.gd script, update the skin variable so it's set in the `_ready` function (as now the parent class has its reference).

```
func _ready() -> void:
    skin = $PlayerSkin
```

For skeleton characters, override the skin variable from the parent character class (in the `_ready` function as well)..

```
skin = $Skins.get_children().pick_random
```

Create the `do_squash_and_stretch` function to handle the animation. This function will create a simple tween that changes the characters scale over time.

```
func do_squash_and_stretch(value: float, duration: float):
    var tween = create_tween()
    tween.tween_property(self, "squash_and_stretch", value, duration)
    tween.tween_property(self, "squash_and_stretch", 1.0, duration * 1.8).set_ease(Tween.EASE_OUT)
```

Call the `do_squash_and_stretch` function in the hit function to trigger the animation when the character is hit.

```
func hit(attacking_weapon):
    if not $Timers/HitTimer.time_left:
```



```
...  
do_squash_and_stretch(1.2, 0.2)
```

Shield Flash Effect

The shield flash effect will make the shield flash briefly when the player defends against an attack. This effect will be achieved using a shader.

Creating the Shield Shader

Follow these steps to create the shield flash shader:

- Open one of the shield scenes, such as `round_shield.tscn`.
- Add a new ShaderMaterial to the shield's material overlay.
- Create a new spatial shader and name it `Flash.gdshader`. Save it in the shaders folder.
- Edit the shader script to create uniforms for alpha and color.

```
shader_type spatial;  
  
uniform float alpha : hint_range(0.0, 1.0) = 0.0;  
uniform vec3 color : source_color = vec3(1.0);  
  
void fragment() {  
    ALBEDO = color;  
    ALPHA = alpha;  
}
```

1. Apply the shader to all shields by creating a material overlay and loading the `Flash.gdshader` shader.

Implementing the Shield Flash Logic

To implement the shield flash logic, follow these steps:

- Open the `shield.gd` script.
- Create the flash function to handle the flash effect using a tween.

```
func flash():  
    var tween = create_tween()  
    tween.tween_method(_flash, 0.0, 1.0, 0.2)  
    tween.tween_method(_flash, 1.0, 0.0, 0.4)
```

Create the `_flash` function to set the shader parameter for alpha.

```
func _flash(value):  
    get_child(0).get_child(0).material_overlay.set_shader_parameter('alpha', value)
```

In the `character.gd` script, call the flash function on the shield when the character is hit and defending.

```
func hit(attacking_weapon):  
    if not $Timers/HitTimer.time_left:
```



```
if current_shield and defending:  
    ...  
    current_shield.flash()
```

By following these steps, you will have implemented a squash and stretch hit animation for characters and a flashing effect for shields when defending. These visual enhancements will make the game more engaging and provide clear feedback to the player.

Troubleshooting

If you encounter any issues, ensure that:

- The character and shield scenes are correctly set up with the appropriate scripts and materials.
- The shader is correctly applied to the shield materials.
- The collision layers and masks are correctly configured for the characters and shields.

With these implementations, your game will have more dynamic and visually appealing combat feedback. Happy coding!



In this lesson, we will focus on adding two crucial features to our game: health management for characters and sound effects. Specifically, we will implement health for both the player and the skeleton enemies, including a death animation for the skeletons. Additionally, we will integrate sound effects to enhance the game's immersive experience. Let's dive into the steps required to achieve these goals.

Adding Health to Characters

First, we need to add a health variable to our character script. This variable will track the health points of each character.

- Open the character script (character.gd).
- Add a new variable for health:

```
var health: int = 5
```

For the skeleton, we will set the initial health to 3 within the `_ready` function:

```
func _ready() -> void:  
    # Existing code...  
    health = 3
```

Updating Health on Hit

Next, we need to update the health whenever a character gets hit. This logic will be implemented inside the `hit` function.

- Modify the `hit` function to subtract the damage from the character's health. We will also add a check now so we can deal with character death:

```
func hit(attacking_weapon):  
    if not $Timers/HitTimer.time_left:  
        if current_shield and defending:  
            health -= attacking_weapon.damage * current_shield.defense  
            current_shield.flash()  
        else:  
            health -= attacking_weapon.damage  
            $Timers/HitTimer.start()  
            do_squash_and_stretch(1.2, 0.2)  
  
    if health <= 0:  
        death_logic()
```

Implementing Death Logic

We need to define what happens when a character's health reaches zero. This will be handled by the `death_logic` function.

- Create a `death_logic` function in the character script. We won't give it functionality, as we'll implement this in our children classes:



```
func death_logic():  
    pass
```

For the player, we will override the death_logic function to quit the game:

- In the player script (player.gd), override the death_logic function:

```
func death_logic():  
    get_tree().quit()
```

Adding Death Animation for Skeletons

For the skeletons, we will add a death animation using the animation tree. Open up the Animation tree and add a new **Animation** node. This one should pull one of the death animations available from the skeleton.

After, add a Blend2 node and rename it to “DeathBlend”. The OneShot attack animation can then be connected to the **in** node of the Blend2 node, and the Death Animation node to the **blend** node. You can then connect the Blend2 node to the Output to make everything work together.

Death Logic for Skeleton

Let’s now manage the actual logic for the skeleton.

- Open the skeleton script (skeleton.gd).
- Create the death_logic function to handle the death animation using a tween. We also want to make sure to disable collisions and the timer to make sure the skeleton can’t interact with the game once dead.

```
func death_logic():  
    $CollisionShape3D.disabled = true  
    $Timers/AttackTimer.stop()  
    var tween = create_tween()  
    tween.tween_method(_death_change, 0.0, 1.0, 0.25)
```

Create the _death_change function to update the blend amount for the death animation:

```
func _death_change(value):  
    $AnimationTree.set("parameters/DeathBlend/blend_amount", value)
```

Finally, ensure that the skeleton stops attacking and becomes non-interactable after death:

```
func _physics_process(delta: float) -> void:  
    if health > 0:  
        move_to_player(delta)  
        if not is_on_floor():  
            apply_gravity(fall_gravity, delta)
```



```
else:
    velocity.y = 0
move_and_slide()
attack_logic()
```

Fixing the Debugger Warning

Lastly, we need to address a debugger warning about an unused parameter. In the player script, add an underscore to the `_input` function parameter to suppress the warning:

```
func _input(_event: InputEvent) -> void:
    ...
```

By following these steps, you will have successfully implemented health management and death logic for both the player and skeleton characters, as well as addressed any debugger warnings. This completes the core game logic for our project. In the next lesson, we will add some audio to our game.



In this lesson, we will focus on adding audio to our game to enhance the player's experience. Audio cues are essential for providing feedback to the player and making the game more immersive. We will add sound effects for various actions, such as hitting an opponent, using weapons, and defending with a shield.

Adding Hit Sound

First, let's add a sound effect for when the player gets hit. Follow these steps:

1. Inside the Character class, add an `AudioStreamPlayer3D` node and rename it to `HitSound`.
2. Drag the audio file named `Impact.ogg` into the `HitSound` node.
3. Adjust the volume by reducing it by 10dB to ensure it's not too loud.
4. To play this sound, modify the hit function in the Character script as follows:

```
func hit(attacking_weapon):
    if not $Timers/HitTimer.time_left:
        if current_shield and defending:
            health -= attacking_weapon.damage * current_shield.defense
            current_shield.flash()
        else:
            health -= attacking_weapon.damage
            $HitSound.play()
    $Timers/HitTimer.start()
    do_squash_and_stretch(1.2, 0.2)

    if health <= 0:
        death_logic()
```

Adding Weapon Sounds

Next, we will add sound effects for each weapon. The steps are as follows:

1. Add an `AudioStreamPlayer` node to the weapon and name it `AttackSound`.
2. In the weapon.gd script, add a new function `func set_sound(audio)` to set the audio stream:

```
func set_sound(audio):
    $AttackSound.stream = audio
```

3. Add another function `func play_audio()` to play the audio:

```
func play_audio():
    $AttackSound.play()
```

4. In the Character script, modify the `equip` function to set the sound for the current weapon using the data from our dictionary:

```
func equip(data, slot):
    for child in slot.get_children():
        child.queue_free()
    var item_scene = data['scene'].instantiate()
    slot.add_child(item_scene)
    if data['type'] == 'weapon':
        item_scene.setup(data['animation'], data['damage'], data['range'], self)
```



```
attack_animation.animation = data['animation']
current_weapon = item_scene
current_weapon.set_sound(data['audio'])
if data['type'] == 'shield':
    item_scene.defense = data['defense']
current_shield = item_scene
```

5. Finally, modify the `ability_logic` function to play the weapon sound when attacking:

```
func ability_logic():
    defending = Input.is_action_pressed("defend")
    if Input.is_action_just_pressed("attack"):
        if not attacking:
            $AnimationTree.set("parameters/AttackOneShot/request", AnimationNodeOneShot.ONE_SHOT_REQUEST_FIRE)
            attacking = true
            current_weapon.play_audio()
```

Adding Shield Sound

Lastly, we will add a sound effect for when the player defends with a shield. Follow these steps:

1. Add an `AudioStreamPlayer` node to the shield and name it `HitSound`.
2. Set the audio file to `metal_button_press_01.wav` and reduce the volume by 10dB.
3. In the `shield.gd` script, add a function `func play_audio()` to play the sound:

```
func play_audio():
    $HitSound.play()
```

4. In the `Character` script, modify the `hit` function to play the shield sound when defending:

```
func hit(attacking_weapon):
    if not $Timers/HitTimer.time_left:
        if current_shield and defending:
            health -= attacking_weapon.damage * current_shield.defense
            current_shield.flash()
            current_shield.play_audio()
        else:
            health -= attacking_weapon.damage
            $HitSound.play()
            $Timers/HitTimer.start()
            do_squash_and_stretch(1.2, 0.2)

    if health <= 0:
        death_logic()
```

By following these steps, you will have successfully added sound effects for hitting, using weapons, and defending with a shield. These audio cues will make the game more engaging and provide better feedback to the player.



In this project overview, we will recap the key concepts and achievements covered in this course.

Key Achievements

- **Player Character Creation:** We successfully created a main player character that responds to player input, allowing for dynamic gameplay.
- **Animation Management:** We utilized animation trees to manage a variety of animations efficiently.
- **Enemy Randomization:** We implemented randomized skeletons with randomized weapons, adding diversity to the game's enemies.
- **Code Simplification with Inheritance:** We employed inheritance to share logic between the player character and the skeletons, simplifying our codebase.

Additional Topics Covered

1. **Raycasting:** We explored the use of raycasting for various in-game mechanics.
2. **Advanced Jumping Logic:** We implemented a sophisticated jumping logic to enhance the player's movement.
3. **Data-Driven Item Creation:** We used data to create items such as weapons and shields, allowing for flexible and scalable item management.

Upcoming Topics

In the next part of the series, we will focus on creating an inventory system for the player. This will allow the player to see and equip different weapons more elegantly. Additionally, we will add hearts to visually represent the player's health. Stay tuned for these exciting updates!

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

You can also download all project files from the **Course Files** tab via the project files or downloadable PDF summary.

global.gd

Found in folder: **/Global**

This code defines three dictionaries: ``weapons``, ``shields``, and ``style``, which store data for different types of game items. Each dictionary contains key-value pairs where the key is the item's name and the value is another dictionary with properties specific to that item, such as damage, defense, scene, animation, and audio. The ``preload`` function is used to load 3D scenes and audio files associated with each item.

extends Node

```
var weapons: Dictionary = {
  'dagger':
  {
    'type': 'weapon',
    'damage': 1,
    'scene': preload("res://scenes/weapons/dagger.tscn"),
    'animation': '1H_Melee_Attack_Stab',
    'range': 1.2,
    'audio': preload("res://audio/dagger_sound.wav")
  },
  'sword':
  {
    'type': 'weapon',
    'damage': 2,
    'scene': preload("res://scenes/weapons/sword.tscn"),
    'animation': '1H_Melee_Attack_Slice_Horizontal',
    'range': 1.5,
    'audio': preload("res://audio/sword_sound.wav")
  },
  'axe':
  {
    'type': 'weapon',
    'damage': 3,
    'scene': preload("res://scenes/weapons/axe.tscn"),
    'animation': '2H_Melee_Attack_Spin',
    'range': 1.3,
    'audio': preload("res://audio/axe_sound.wav")
  },
  'staff':
  {
    'type': 'weapon',
    'damage': 1,
    'scene': preload("res://scenes/weapons/staff.tscn"),
    'animation': '2H_Melee_Attack_Slice',
    'range': 2.1,
    'audio': preload("res://audio/staff_sound.wav")
  }
}
```

```
    },
}
var shields: Dictionary = {
    'square': {
        'type': 'shield',
        'defense': 0.8,
        'scene': preload("res://scenes/shields/square_shield.tscn")
    },
    'round': {
        'type': 'shield',
        'defense': 0.9,
        'scene': preload("res://scenes/shields/round_shield.tscn")
    },
    'spike': {
        'type': 'shield',
        'defense': 0.6,
        'scene': preload("res://scenes/shields/spike_shield.tscn")
    },
}
var style: Dictionary = {
    'sunglasses': {
        'type': 'style',
        'scene': preload("res://scenes/style/sunglasses.tscn"),
    },
    'starglasses': {
        'type': 'style',
        'scene': preload("res://scenes/style/starglasses.tscn"),
    },
    'duckhat': {
        'type': 'style',
        'scene': preload("res://scenes/style/duck_hat.tscn"),
    },
    'tophat': {
        'type': 'style',
        'scene': preload("res://scenes/style/tophat.tscn"),
    },
}
```

camera_controller.gd

Found in folder: **/scenes/characters**

This script controls the rotation of a 3D camera arm based on user input. It uses joystick or gamepad input to rotate the camera, with adjustable acceleration and limits on the rotation. The script also includes commented-out code for capturing mouse input and rotating the camera based on mouse movement.

extends SpringArm3D

```
@export var min_limit_x: float = -0.8
@export var max_limit_x: float = 0.2
@export var horizontal_acceleration: float = 2
@export var vertical_acceleration: float = 1
```

```
@export var mouse_acceleration: float = 0.005

func _process(delta: float) -> void:
    var joy_dir = Input.get_vector("pan_left", "pan_right", "pan_up", "pan_down")
    var joy_dir_accelerated = joy_dir * delta * Vector2(horizontal_acceleration, vertical_acceleration)
    rotate_from_vector(joy_dir_accelerated)

#func _ready() -> void:
#    #Input.mouse_mode = Input.MOUSE_MODE_CAPTURED
#
#func _input(event: InputEvent) -> void:
#    #if event is InputEventMouseMotion:
#        #var mouse_dir_accelerated = event.relative * mouse_acceleration
#        #rotate_from_vector(mouse_dir_accelerated)
#    #if Input.is_action_just_pressed("ui_cancel"):
#        #get_tree().quit()

func rotate_from_vector(v: Vector2):
    if v.length() == 0 : return
    rotation.y -= v.x
    rotation.x -= v.y
    rotation.x = clamp(rotation.x, min_limit_x, max_limit_x)
```

character.gd

Found in folder: **/scenes/characters**

This script defines the behavior of a character in a 3D game, including movement, jumping, attacking, defending, and taking damage. It also manages the character's animation states, equipment, and health. The script uses various variables and functions to control the character's actions and reactions to different situations.

```
class_name Character
extends CharacterBody3D

@export var base_speed: float = 4
@export var run_speed: float = 6
@export var defend_speed: float = 2
var movement_input: Vector2

@export var jump_height: float = 2.25
@export var jump_time_to_peak: float = 0.4
@export var jump_time_to_descent: float = 0.3

@onready var jump_velocity: float = ((2.0 * jump_height) / jump_time_to_peak) * -1.0
@onready var jump_gravity: float = ((-2.0 * jump_height) / (jump_time_to_peak * jump_time_to_peak)) * -1.0
@onready var fall_gravity: float = ((-2.0 * jump_height) / (jump_time_to_descent * jump_time_to_descent)) * -1.0

@onready var move_state_machine = $AnimationTree.get("parameters/MoveStateMachine/playback") as AnimationNodeStateMachinePlayback
```



```
@onready var attack_animation = $AnimationTree.get_tree_root().get_node('AttackAnimation') as AnimationNodeAnimation

var skin: Node3D
var attacking: bool = false
var defending: bool = false:
    set(value):
        if not defending and value:
            defend_toggle(true)
        if defending and not value:
            defend_toggle(false)
        defending = value
var current_weapon: Node3D
var current_shield: Node3D
var squash_and_stretch: float = 1.0:
    set(value):
        squash_and_stretch = value
        var negative = 1.0 + (1.0 - squash_and_stretch)
        skin.scale = Vector3(negative, squash_and_stretch, negative)
var health: int = 5

func defend_toggle(forward: bool):
    var tween = create_tween()
    tween.tween_method(_defend_change, 1.0 - float(forward), float(forward), 0.25)

func _defend_change(value):
    $AnimationTree.set("parameters/DefendBlend/blend_amount", value)

func apply_gravity(gravity, delta):
    velocity.y -= gravity * delta

func set_move_state(state_name: String):
    move_state_machine.travel(state_name)

func equip(data, slot):
    for child in slot.get_children():
        child.queue_free()
    var item_scene = data['scene'].instantiate()
    slot.add_child(item_scene)
    if data['type'] == 'weapon':
        item_scene.setup(data['animation'], data['damage'], data['range'], self)
        attack_animation.animation = data['animation']
        current_weapon = item_scene
        current_weapon.set_sound(data['audio'])
    if data['type'] == 'shield':
        item_scene.defense = data['defense']
        current_shield = item_scene

func _on_animation_tree_animation_finished(anim_name: StringName) -> void:
    if 'Attack' in anim_name:
```

```
    attacking = false

func attack_logic():
    if attacking:
        var collider = current_weapon.get_collider()
        if collider and collider != self and "hit" in collider:
            collider.hit(current_weapon)

func hit(attacking_weapon):
    if not $Timers/HitTimer.time_left:
        if current_shield and defending:
            health -= attacking_weapon.damage * current_shield.defense
            current_shield.flash()
            current_shield.play_audio()
        else:
            health -= attacking_weapon.damage
            $HitSound.play()
            $Timers/HitTimer.start()
            do_squash_and_stretch(1.2, 0.2)

    if health <= 0:
        death_logic()

func death_logic():
    pass

func do_squash_and_stretch(value: float, duration: float):
    var tween = create_tween()
    tween.tween_property(self, "squash_and_stretch", value, duration)
    tween.tween_property(self, "squash_and_stretch", 1.0, duration * 1.8).set_ease(Tween.EASE_OUT)
```

player.gd

Found in folder: **/scenes/characters**

This script controls the player character's movement, actions, and abilities. It handles input for movement, jumping, attacking, and defending, and updates the character's state and animation accordingly. The script also manages the character's equipment, such as switching between different weapons and shields.

extends Character

```
@export var acceleration: float = 8
@export var deceleration: float = 4
@onready var camera = $CameraController/Camera3D
```

```
var weapons = [Global.weapons['sword'], Global.weapons['dagger'], Global.weapons['staff']]
```



```
var weapon_index: int
var shields = [Global.shields['round'], Global.shields['square'], Global.shields['spike']]
var shield_index: int
var styles = [Global.style['duckhat']]
var style_index: int

func _ready() -> void:
    skin = $PlayerSkin
    equip(weapons[weapon_index], $PlayerSkin/Rogue/Rig/Skeleton3D/RightHand)
    equip(shields[shield_index], $PlayerSkin/Rogue/Rig/Skeleton3D/LeftHand)
    #equip(styles[style_index], $PlayerSkin/Rogue/Rig/Skeleton3D/Head)

func _physics_process(delta: float) -> void:
    move_logic(delta)
    jump_logic(delta)
    ability_logic()
    move_and_slide()
    attack_logic()

func _input(_event: InputEvent) -> void:
    if Input.is_action_just_pressed("switch_weapon"):
        weapon_index = posmod(weapon_index + 1, weapons.size())
        equip(weapons[weapon_index], $PlayerSkin/Rogue/Rig/Skeleton3D/RightHand)
    if Input.is_action_just_pressed("switch_shield"):
        shield_index = posmod(shield_index + 1, shields.size())
        equip(shields[shield_index], $PlayerSkin/Rogue/Rig/Skeleton3D/LeftHand)

func move_logic(delta):
    movement_input = Input.get_vector("left", "right", "forward", "backward").rotated(-camera.global_rotation.y)
    var vel_2d = Vector2(velocity.x, velocity.z)

    if movement_input != Vector2.ZERO:
        var speed = run_speed if Input.is_action_pressed("run") else base_speed
        speed = defend_speed if defending else speed
        vel_2d += movement_input * speed * delta * acceleration
        vel_2d = vel_2d.limit_length(speed)
        velocity.x = vel_2d.x
        velocity.z = vel_2d.y
        var target_angle = -movement_input.angle() + PI / 2
        skin.rotation.y = rotate_toward(skin.rotation.y, target_angle, delta * 6)
        set_move_state('Running_A')
    else:
        vel_2d = vel_2d.move_toward(Vector2.ZERO, base_speed * delta * deceleration)
        velocity.x = vel_2d.x
        velocity.z = vel_2d.y
        set_move_state('Idle')

func jump_logic(delta):
    if is_on_floor():
```



```
if Input.is_action_just_pressed("jump"):
    velocity.y = -jump_velocity
else:
    set_move_state('Jump_Idle')
var gravity = jump_gravity if velocity.y > 0.0 else fall_gravity
apply_gravity(gravity, delta)

func ability_logic():
    defending = Input.is_action_pressed("defend")
    if Input.is_action_just_pressed("attack"):
        if not attacking:
            $AnimationTree.set("parameters/AttackOneShot/request", AnimationNodeOneShot.ONE_SHOT_REQUEST_FIRE)
            attacking = true
            current_weapon.play_audio()

func death_logic():
    get_tree().quit()
```

skeleton.gd

Found in folder: **/scenes/characters**

This code controls an enemy character's behavior, including movement, attack logic, and death animation. The enemy moves towards the player when within a certain notice radius and attacks when close enough. The enemy's movement and attack behaviors are influenced by variables such as speed, attack radius, and notice radius, which can be adjusted in the editor.

```
extends Character
```

```
@onready var player = get_tree().get_first_node_in_group('Player')
@export var notice_radius: float = 30
@export var speed: float = 3
var attack_radius: float = 3
var rng = RandomNumberGenerator.new()

func _ready() -> void:
    for child in $Skins.get_children():
        child.hide()
    skin = $Skins.get_children().pick_random()
    skin.show()
    $AnimationTree.anim_player = "../Skins/" + skin.name + '/AnimationPlayer'
    var random_weapon = Global.weapons[Global.weapons.keys().pick_random()]
    equip(random_weapon, skin.get_node('Rig/Skeleton3D/RightHand'))
    attack_radius = current_weapon.radius
    health = 3

func _physics_process(delta: float) -> void:
    if health > 0:
```

```
move_to_player(delta)
if not is_on_floor():
    apply_gravity(fall_gravity, delta)
else:
    velocity.y = 0
move_and_slide()
attack_logic()

func move_to_player(delta):
    if position.distance_to(player.position) < notice_radius:
        var target_dir = (player.position - position).normalized()
        var target_vec2 = Vector2(target_dir.x, target_dir.z)
        var target_angle = -target_vec2.angle() + PI/2
        rotation.y = rotate_toward(rotation.y, target_angle, delta * 6)
        if position.distance_to(player.position) > attack_radius:
            velocity = Vector3(target_vec2.x, 0, target_vec2.y) * speed
            set_move_state('Running_A')
        else:
            velocity = Vector3.ZERO
            set_move_state('Idle')

func _on_attack_timer_timeout() -> void:
    $Timers/AttackTimer.wait_time = rng.randf_range(2.0, 3.5)
    if position.distance_to(player.position) < attack_radius:
        $AnimationTree.set('parameters/AttackOneShot/request', AnimationNodeOneShot.ONE_SHOT_REQUEST_FIRE)
        attacking = true
        current_weapon.play_audio()

func death_logic():
    $CollisionShape3D.disabled = true
    $Timers/AttackTimer.stop()
    var tween = create_tween()
    tween.tween_method(_death_change, 0.0, 1.0, 0.25)

func _death_change(value):
    $AnimationTree.set("parameters/DeathBlend/blend_amount", value)
```

shield.gd

Found in folder: **/scenes/shields**

This script extends the Node3D class and represents a shield equipment with a defense value. It has a flash function that temporarily changes the alpha value of a material overlay, creating a flashing effect. The script also has a play_audio function that plays a sound effect when called.

extends Node3D

```
var defense: float
const equipment_type = 'Shield'
```

```
func flash():
    var tween = create_tween()
    tween.tween_method(_flash, 0.0, 1.0, 0.2)
    tween.tween_method(_flash, 1.0, 0.0, 0.4)

func _flash(value):
    get_child(0).get_child(0).material_overlay.set_shader_parameter('alpha', value)

func play_audio():
    $HitSound.play()
```

weapon.gd

Found in folder: **/scenes/weapons**

This script defines a 3D node that represents a weapon in the game. It stores properties such as animation, damage, and radius, which are set up through the `setup` function. The script also provides methods to get the collider, set and play audio for the weapon's attack sound.

```
extends Node3D

const equipment_type = 'Weapon'
var animation: String
var damage: int
var parent
var radius: float

func setup(weapon_animation, weapon_damage, weapon_radius, weapon_parent):
    animation = weapon_animation
    damage = weapon_damage
    parent = weapon_parent
    radius = weapon_radius

func get_collider():
    return $RayCast3D.get_collider()

func set_sound(audio):
    $AttackSound.stream = audio

func play_audio():
    $AttackSound.play()
```

flash.gdshader

Found in folder: **/shaders**



This shader sets the color and transparency of a 3D object. It uses two variables: “color” to set the object’s color, and “alpha” to set its transparency level between 0 (fully transparent) and 1 (fully opaque). The shader applies these values to the object’s material in the fragment function.

```
shader_type spatial;

uniform float alpha: hint_range(0.0, 1.0) = 0.0;
uniform vec3 color: source_color = vec3(1.0);

void fragment() {
    ALBEDO = color;
    ALPHA = alpha;
}
```